Advanced computerarchitecture
Exercise 3 Report

(B-KUL-JPI30R)

Wout Van Steenbergen

March 4, 2024



Session Date: February 26, 2024

Instructor: Alex Marinšek

1 Introduction

In this report the concepts of striding, thread divergence and warps within CUDA and the NVIDIA GPU's will be explained. Image inversion is used as example, focusing on the manipulation of RGB color components within digital images. The impact of parallelization strategies on computation time will be investigate by adjusting the number of threads within full or partial warps. Additionally, a conditional operation for the Red (R) color component is introduced to examine how thread divergence affects performance. Both help highlighting the balance between parallelization benefits and the overhead of thread divergence.

2 striding

"stride" refers to the step size or interval used when accessing elements in a sequence, such as an array or memory. Its A technique that is used when there's not a one-to-one correspondence between the number of processing threads and the elements in a data set. It describes a situation where individual threads are responsible for computing multiple elements in an array by cycling through the array with a step equal to the total number of threads, known as the stride size. In the case of this report it means that a single thread is responsible for multiple pixels. Proper management and optimization of stride can lead to an overall performance improvements in parallel computing tasks.

3 warps

A "warp" is the basic unit of execution. It consists of a group of threads that execute the same instruction at the same time on different data. The size of a warp is fixed for a given GPU architecture. for most NVIDIA GPUs, a warp consists of 32 threads. The threads in a warp start together at the same program address, but they have their own instruction counter and register state and can diverge, meaning they can follow different execution paths based on their data. However, divergent paths within a warp are serialized, which can affect performance since some threads may be idle while others are executing different instructions. When a warp is fully utilized, meaning all 32 threads in the warp are active and executing instructions, the computational efficiency is maximized leading to optimal performance and lower computation time for tasks. This is because the GPU's warp scheduler can dispatch instructions to all threads in the warp without any idle cycles. Partial warps occur when the number of threads in a block is not a multiple of the warp size. The GPU will executes warp-sized instructions with some threads not doing useful work. This leads to wasted computational resources and increases computation time since the GPU is executing instructions for threads that are not contributing.

- Elements 64, threads 64 = two full warps. Since each thread in the warp is active and can be processed in parallel, all 32 cores are used for each warp.

- Elements 50, threads 50 = one full - and one partial warp. For the full warp, all 32 cores are utilized. However, for the partial warp, only 18 of the 32 cores are utilized, as there are only 18 threads in the second warp. This means 14 cores are idle and not contributing to the computation during the execution of the partial warp.

4 thread divergence

Thread divergence is a a situation where threads within the same warp follow different execution paths due to conditional branching. When a conditional branch (e.g., an if-else statement) is encountered, and not all threads in a warp agree on which path to take, the warp is divided into different execution paths. The GPU executes each path serially, one after the other, rather than in parallel. This means that while some threads are executing one branch of the condition, the rest are idle, waiting for their turn to execute their branch of the code. This serialization of divergent branches can lead to decreased performance because the computational resources are not being fully utilized. Thread divergence can be caused by various factors like conditional statements (if-else or switch statements) or loops where the number of iterations varies between threads.

5 Results

The three different Kernels where implemented that revert an image based on the following calculation $[x = 255 - x]$ to help visualize the effect of full and partial warps. the first kernel is the simple inversion. The second kernel invokes thread divergence by calculating the red component as following $[\ x = (x * 25/100) * 10, x \in (100, 200)\ ]$. The last one transforms the input array structured as a sequence of RGB triplets [RGB RGB RGB ...] into an array where all red, green and blue components are grouped together[RRR...GGG...BBB...]. The result are visible in figure 1.

In the simple inversion kernel, the absence of any conditionals or divergent code paths means that all threads in a warp can execute in lockstep, which refers to the synchronous execution of multiple threads. this means that every thread within a warp executes the same instruction at the same time, which is the most efficient mode of execution on a GPU. This is why you see a dip in time at specific thread counts (928, 960 & 992), which are multiples of 32.

For the second kernel, which includes thread divergence, you do not see these dips because the divergence disrupts the warp execution. When threads in a warp follow different execution paths they cannot execute in lockstep. This serializes the execution of divergent paths and prevents the kernel from taking full advantage of warp-level parallelism.

The increase in execution time for the third kernel can be can be explained by the code implementation. In the first two kernels, each thread is responsible for processing the inversion of an entire pixel, which includes the red, green, and blue components together. contrarily in the third kernel, each thread is tasked with computing the inversion for only one color component of a pixel instead of all three.

In Figure 2, The impact is observed on computation time introduced by an additional kernel leveraging two-dimensional (2D) blocks and stride. Notably, there are again dips in time, attributed to the full warps. However, the increased time cost associated with 2D blocks could be explained by the necessity of mapping the grid to the GPU's physical architecture, which is inherently one-dimensional.

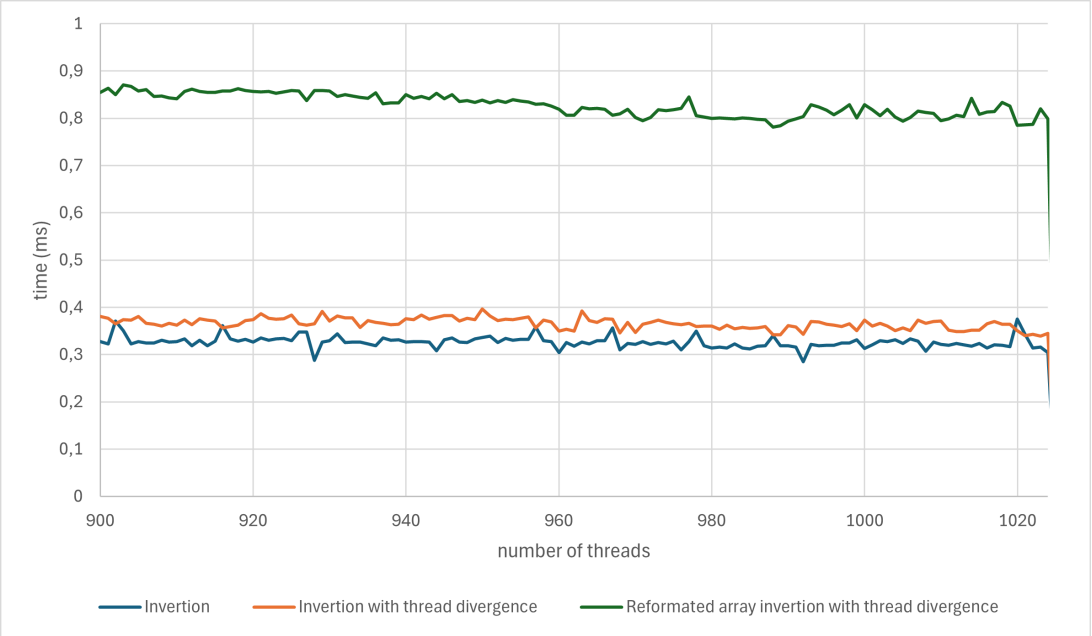


Figure 1: processing time (ms) of kernels for n number of threads

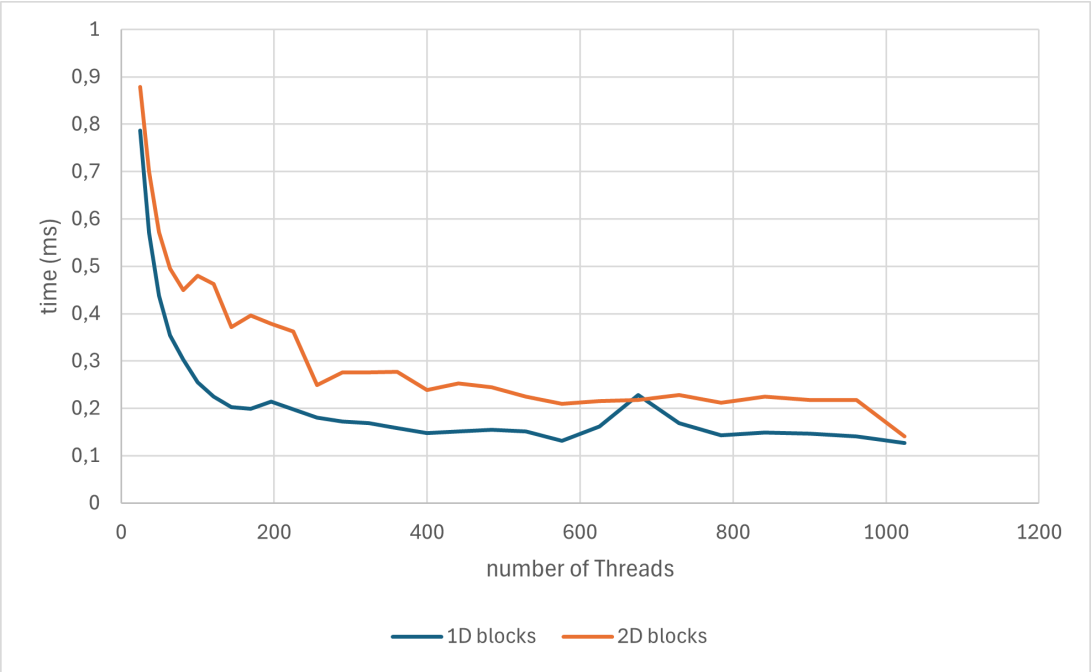


Figure 2: processing time (ms) of multiple dimension blocks for n number of threads